Complementarity is Abundant, Predictability is Not the Problem: Decomposing Why LLM Routers Gain So Little

Anonymous submission

Abstract

LLM routers promise frontier quality at small-model cost. We ask where the promised gain actually goes. We decompose a router's achievable gain over the correct baseline—a *cost-matched, query-independent* policy—into three multiplicative and additive terms: *complementarity* κ (how much oracle headroom the model set offers), *predictability* ρ (what fraction of that headroom is a function of the query at all), and *estimation error* ε (what a fitted router fails to capture). We prove $\rho = 1$ exactly when outcomes are deterministic given the query, give a distribution-free ceiling $\sqrt{\beta(1-\beta)}\,\mathrm{sd}(\delta)$ on every router, and show $\mathrm{sd}(\delta)$ is identified only from *repeated* generations—which no public routing dataset provides. On RouterBench (36,494 prompts $\times$ 11 models, all 55 model pairs $\times$ 8 benchmark families) complementarity is abundant: median $\kappa = 12.11$ pp of oracle headroom at matched cost. Yet the best of four router families captures 0.59 pp, or $\rho_{\text{realised}} = 4.6\%$ of it. We then pre-registered and *refuted* our own explanation. Using $k = 3$ replicate generations of 5,000 prompts across three models—data we generated because no public dataset replicates—we find per-item difficulty is highly *reliable* (0.77–0.98 of outcome variance is stable between-item signal, implying a Bayes-optimal AUC of 0.95–0.99), so $\rho \approx 1$ and unpredictability is *not* the bottleneck. The entire gap is ε: which model will succeed is a stable property of the item that routers cannot read off the prompt. In *AUC* that gap is immovable: nine router families—TF-IDF through a prompted 70B model and an *end-to-end fine-tuned* encoder—all land in 0.64–0.72, no family's advantage over a frozen MiniLM+logistic baseline survives a paired bootstrap with Holm correction, and a learning curve over 250 to 29,000 items extrapolates to an asymptote of 0.74. But AUC turns out to be the wrong thing to watch, and the decomposition is what reveals it. The highest-AUC router we built has the *worst* matched-cost gain (-1.31 pp, worse than ignoring the query); conversely, unfreezing the encoder buys the same $+0.005$ AUC on both our data and RouterBench while *tripling* the deployable gain at RouterBench scale, from $\rho_{\text{realised}} = 9.7\%$ to 32.6%. So ε is large but not immovable, and reducing it looks like a representation-learning problem that per-model AUC cannot see. Two thirds of κ nonetheless remains unclaimed, and the whole analysis replicates on RouterBench's independent 5-shot release ($\kappa = 11.44$ pp). Routing's binding constraint is generalisation, not noise and not model redundancy—and it is being measured with a metric that does not track it.

1 Introduction

A cascade or router sends easy queries to a cheap model and hard ones to an expensive one, then reports savings against always using the expensive model. That comparison is nearly uninformative, because always-cheap and any fixed random mixture also beat always-expensive on cost while looking at nothing. The bar a router must clear is the best *query-independent* policy at the *same expected cost*. Reported gains against always-frontier are not evidence that reading the query helped.

Fixing the baseline is necessary but not sufficient, because it tells you *that* a gain is small without telling you *why*. Three very different worlds produce the same small number. The models might be redundant, so there is nothing to win. The outcome might be a coin flip given the query, so nothing is winnable by any router. Or the information might be there and the router might simply fail to extract it. The first is a model-selection problem, the second a property of the task, the third an engineering problem—and the correct response differs completely.

Contributions.

1. **A decomposition that separates those three worlds** (§2). Achievable gain = $\rho(b)\,\kappa(b)$, realised gain = $\rho(b)\kappa(b) - \varepsilon$. We prove the ladder $S(b) \le A^{\dagger}(b) \le A^{*}(b)$, closed forms for two models, a distribution-free ceiling on *every* router, the exact Bayes-optimal AUC implied by a difficulty distribution, and—the key structural point—that $\rho < 1$ is *exactly* outcome randomness given the query, so an oracle measured on single generations overstates attainable headroom. All five are validated numerically against brute-force LP and Monte-Carlo references (14/14 checks, Appendix B).
2. **Complementarity is abundant and routers capture almost none of it** (§3). On RouterBench, median $\kappa = 12.11$ pp $[11.15, 12.92]$ across 55 model pairs × 8 benchmark families, of which the best of four router families realises 4.6% $[3.6\%, 5.9\%]$. The direction is consistent (49/55 pairs positive, 13/55 significant under Holm), so routing does work—it just recovers a twentieth of what is there.
3. **A pre-registered refutation of our own hypothesis** (§4). We predicted low predictability and tested it with $k = 3$ replicate generations of 5,000 prompts × 3 models. The data says the opposite: difficulty is reliable, $\rho \approx 1$, and our ceiling bound is vacuous at these effect sizes. The refutation sharpens the claim rather than weakening it: the whole gap is ε.
4. **That gap is stubborn in AUC and partly tractable in deployable gain** (§5). Nine router families—TF-IDF, MiniLM with four model classes, a prompted 70B model zero-shot and 4-shot, and two *end-to-end fine-tuned* encoders—all land in 0.64–0.72 AUC, none beats the frozen baseline under a paired bootstrap with Holm correction, and a learning curve over 250 to 29,000 items extrapolates to 0.74. Yet at RouterBench scale an end-to-end fine-tuned encoder captures 32.6% of κ against 9.7% for the frozen representation refit on the same items—for the same +0.005 AUC.
5. **Per-model AUC is not a proxy for router value** (§5). Besides failing to register that 3.4×, AUC gets the sign wrong: the prompted 4-shot router has the best AUC of any rung (0.710) and the *worst* matched-cost gain (−1.31 pp, worse than ignoring the query). AUC is invariant to the monotone rescaling of per-model scores that a matched-cost policy depends on, so a router can rank items well within each model and still make the wrong *cross-model* comparison on every one. The literature reports AUC or accuracy-at-a-budget almost universally.

We claim no new router, architecture or algorithm, and we do not claim routing cannot work. κ and ρ are properties of a (workload, model set) pair; the contribution is the instrument that says which term is binding.

2 The decomposition

A query $x \sim \mathcal{D}$; K models; running model m on x yields utility $u_m(x) \in [0,1]$ (graded correctness) and cost $c_m(x) \ge 0$. Both are random *given* x: decoding is stochastic, and measurably so even at temperature 0 (§4). Write $\eta_m(x) = \mathbb{E}[u_m(x) \mid x]$ and $\gamma_m(x) = \mathbb{E}[c_m(x) \mid x]$. A policy assigns x to a model with probabilities $z_m(x)$, and has value $A(z) = \mathbb{E}[\sum_m z_m(x) u_m(x)]$ and cost $C(z) = \mathbb{E}[\sum_m z_m(x) c_m(x)]$. Three nested classes, distinguished only by what they may look at, with optima at budget b:

$S(b)$ (query-independent), $A^{\dagger}(b)$ (router: any function of x), $A^{*}(b)$ (oracle: sees realised u, c).

Proposition 1 (Ladder). *$S(b) \le A^{\dagger}(b) \le A^{*}(b)$, and all three are concave and nondecreasing in b.*

Definition 1. *$\kappa(b) = A^{*}(b) - S(b)$ (complementarity) and $\rho(b) = (A^{\dagger}(b) - S(b))/\kappa(b)$ (predictability), so achievable gain $= \rho(b)\kappa(b)$ and any fitted router $\hat{z}$ realises $A(\hat{z}) - S(b) = \rho(b)\kappa(b) - \varepsilon(\hat{z})$ with $\varepsilon \ge 0$.*

The identity is true by construction; its use is that the three terms are separately measurable and assign blame differently (κ: the model set; ρ: the task; ε: the engineer).

Proposition 2 (What $\rho < 1$ is). *If u_m and c_m are deterministic given x for every m, then $\rho(b) = 1$ for all b. Hence every unit of $\rho < 1$ is outcome randomness given the query.*

Proposition 2 inverts how oracle headroom is normally read. A routing paper's oracle is computed from *one* generation per model per item, so it routes each item to whichever model happened to be right on that

draw. When outcomes are noisy, part of what it collects is a coin flip it saw and no router can see. Published oracle headroom is an upper bound inflated by generation noise.

For $K = 2$ (cheap s, dear l, β = traffic fraction to l), with $\delta(x) = \eta_l(x) - \eta_s(x)$, $D = u_l - u_s$, and CTE_β the mean of the upper β-tail: $S(b) = \mathbb{E}[u_s] + \beta\mathbb{E}[D]$, $A^\dagger(b) = \mathbb{E}[u_s] + \beta\mathrm{CTE}_\beta(\delta)$, $A^*(b) = \mathbb{E}[u_s] + \beta\mathrm{CTE}_\beta(D)$, so $\rho(\beta) = \big(\mathrm{CTE}_\beta(\delta) - \mathbb{E}\delta\big)/\big(\mathrm{CTE}_\beta(D) - \mathbb{E}D\big)$: predictability is the ratio of tail dispersion of the *conditional mean* to that of the *realisation*. In the binary case the gain of an unconstrained oracle over the better single model is exactly $\min(p_{01}, p_{10})$.

Proposition 3 (Router-free ceiling)**.** *For every router at operating fraction β,*

$$A^\dagger(b) - S(b) \;\le\; \sqrt{\beta(1-\beta)}\,\mathrm{sd}(\delta) \;\le\; \tfrac{1}{2}\,\mathrm{sd}(\delta),$$

and the bound is attained.

Proof. For any $z : \mathcal{X} \to [0,1]$ with $\mathbb{E}z = \beta$, $\mathbb{E}[z\delta] - \beta\mathbb{E}[\delta] = \mathrm{Cov}(z, \delta) \le \mathrm{sd}(z)\,\mathrm{sd}(\delta)$ by Cauchy–Schwarz, and a $[0,1]$-valued variable with mean β has variance at most $\beta(1-\beta)$. Tightness: take δ two-valued and z its indicator. □

This is the same covariance object as the cost-accounting correction of Appendix J: routing lives or dies on one covariance, read on two axes.

Proposition 4 (Identification from replicates)**.** *With $k \ge 2$ conditionally i.i.d. replicates per item and independence across models, $\mathrm{Var}(\delta)$ is identified by subtracting a within-item noise term from the observed dispersion of $\hat\eta_l - \hat\eta_s$; the estimator and its unbiasedness are in Appendix A.*

The consequence is the one that matters. κ is computable from any released outcome matrix; ρ is not. Since $\mathrm{Var}(D) = \mathrm{Var}(\delta) + \mathbb{E}[\mathrm{Var}(D \mid x)] \ge \mathrm{Var}(\delta)$, single-generation data bounds $\mathrm{sd}(\delta)$ only loosely—and *every public routing dataset stores one generation per (model, prompt).* So the quantity that decides whether routing can work is not recoverable from the data the field publishes: in simulation the uncorrected estimator inflates $\mathrm{Var}(\delta)$ by 2.33× (Appendix B). This is why we generated replicates rather than relying on RouterBench alone.

Proposition 5 (Bayes-optimal AUC)**.** *With $a = \mathbb{E}[\eta(X)]$ and X, X' i.i.d.,*

$$\mathrm{AUC}^\star = \tfrac{1}{2} + \frac{\mathbb{E}|\eta(X) - \eta(X')|}{4a(1-a)}, \qquad \mathrm{AUC}^\star \le \tfrac{1}{2} + \frac{\mathrm{sd}(\eta)}{2\sqrt{2}\,a(1-a)}.$$

Proposition 5 turns a replicate-estimated quantity into a prediction about a number we have measured with many learners—a check that cannot be tuned. Proofs: Appendix A.

3 Complementarity is abundant; routers capture 4.6% of it

Setup. RouterBench [Hu et al., 2024]: 36,494 prompts × 11 models = 401,434 inference outcomes with per-item graded score and per-item dollar cost, grouped into 8 benchmark families with $n \ge 100$ (MMLU 14,042; HellaSwag 10,042; GSM8K 7,450; ARC 1,470; Winogrande 1,267; Chinese 785; other 931; MBPP 427). 21.1% of score cells are fractional partial credit, so we run the decomposition on graded utilities in $[0,1]$, which needs no threshold. We verified these fractional scores are *not* replicate means—GPT-4 is modal at 0.75 and Mistral-7B at 0.25 with almost no mass at 0 or 1, and only one response is stored per model—so RouterBench cannot support Proposition 4.

We compute $S(b)$ exactly (the upper concave envelope of the per-model cost–utility points) and $A^*(b)$ exactly (the Lagrangian solution of the multiple-choice knapsack LP on *per-item* costs), for all 55 unordered model pairs and for the full 11-model problem, over $\beta \in \{0.1, \ldots, 0.9\}$; headline numbers are at $\beta = 0.5$, where Proposition 3's cap is largest and routing is therefore most favoured. Four router families are fit with 5-fold cross-validation grouped by item and stratified by family, so every router trains on 29,195 items. Routers commit using their predictions and per-model *mean* costs (a deployed router cannot know an item's token count in advance) but are charged *realised* per-item cost.

Table 1: RouterBench, $\beta = 0.5$: medians over 55 model pairs × 8 benchmark families (440 cells), with 95% bootstrap intervals over cells. ρ_{realised} is realised gain divided by κ.

Router	out-of-fold AUC	gain (pp)	ρ_{realised}	cells with gain > 0
TF-IDF + logistic	0.7160	+0.585 [0.442, 0.777]	0.046 [0.036, 0.059]	311/440
MiniLM + logistic	0.7081	+0.455 [0.286, 0.609]	0.034	288/440
MiniLM + boosted trees	0.6772	+0.402 [0.268, 0.544]	0.033	290/440
MiniLM + MLP (256,64)	0.6626	+0.226 [0.138, 0.326]	0.017	269/440
Complementarity κ		**12.11** pp [11.15, 12.92]		

Table 1 is the central empirical result. There is 12.11 pp of oracle headroom over a cost-matched query-independent policy, and the best router takes 0.59 pp of it. The effect is real: 49 of 55 pairs show a positive mean gain, 28 have raw $p < 0.05$, 26 survive Benjamini–Hochberg and 13 survive Holm–Bonferroni at family-wise $\alpha = 0.05$. So this is not a null result about routing; it is a statement about magnitude. Reading gains against always-frontier hides a factor of roughly twenty.

Note also that the MLP is the *worst* of the four despite the most capacity—the same inversion we observe in-house, and the first hint that the binding constraint is not model class.

Replication on the 5-shot release. RouterBench ships a second, independent set of generations in which every prompt carries five in-context exemplars. It was pre-registered as a replication and analysed with identical code: $\kappa = 11.44$ pp [10.66, 12.27], best-router gain +0.640 pp [0.519, 0.790], $\rho_{\text{realised}} = 5.5\%$ [4.4%, 6.8%], with 50 of 55 pairs positive and 11 surviving Holm. Both hypotheses are supported again and the headline ratio moves by about a percentage point of κ. (28 of 36,508 5-shot items are dropped for missing score cells; the 0-shot file has none.)

4 Predictability is not the bottleneck: a refuted hypothesis

We pre-registered the hypothesis that ρ is small—that per-item success is weakly predictable in principle—and a falsification rule for it. To test it we generated $k = 3$ replicates for every (item, model) over 5,000 prompts and three models at temperature 0.7 (45,000 calls) and 364 prompts at temperature 0 (3,276 calls), on HumanEval, MBPP, MMLU and GSM8K with execution- or exact-match grading. **The hypothesis was refuted.**

Call *reliability* the share of per-item outcome variance that is stable between-item signal rather than regeneration noise. Across three models and both sets it runs 0.772–0.983 (lowest the 9B tier at temperature 0, highest GPT-4o), so whether a model answers an item correctly is not a coin flip: 77–98% of the outcome variance is a stable property of the item. The corresponding Proposition 5 estimates under a Beta fit to the replicate-identified moments run $\text{AUC}^\star = 0.948$–1.000, i.e. a Bayes-optimal AUC of 0.95–0.99 against the 0.65–0.72 that nine router families reach (per-set, per-tier values in Table 7, Appendix H). So $\rho \approx 1$: the information a router would need *is* there. Our pre-registered consistency check predicted the replicate-implied $\text{AUC}^\star$ would land within 0.05 of the 0.65–0.68 plateau measured by four learners; it came out 0.99 against 0.68, a miss of 0.31, and is recorded as inconsistent. We included that check because it could embarrass us, and it did.

Correspondingly Proposition 3's ceiling is *vacuous* at these effect sizes: at $\beta = 0.5$ it evaluates to 8.4–15.6 pp against a single-draw κ of 3.3–9.5 pp, a median ratio of 2.82. The inequality is correct and attained on a two-point distribution, but real δ is diffuse, and Cauchy–Schwarz loses exactly that slack. We report this rather than reframing it.

Two things do survive. Noise inflates the reported oracle: a policy allowed to peek at one graded outcome per model—information no deployed router has— captures only 34–55% of the single-draw κ on held-out replicates. And 8–35% of the variance of the observed advantage D is generation noise, the share scaling with output length, from 8% for terse GPT-4o to 35% for a long-reasoning 9B model that changes its graded verdict on 19.8% of items between identical temperature-0 calls.

Table 2: Unfreezing the encoder on RouterBench: 27,735 training items, 11 models, one 7,299-item held-out split, $\kappa = 19.95$ pp at $\beta = 0.5$. All three rows are fitted on identical items and scored by identical code. ρ_{realised} is the share of κ the router actually captures. Exploratory; not pre-registered.

Router	mean AUC	gain (pp)	ρ_{realised}
MiniLM (frozen) + logistic	0.7078	+1.939	9.7%
MiniLM (frozen) + MLP	0.6617	+2.825	14.2%
MiniLM **unfrozen**, fine-tuned end-to-end	**0.7126**	**+6.507**	**32.6**%

5 The binding term is generalisation

If $\rho \approx 1$ and routers capture 4.6% of κ, then ε is essentially the whole story: difficulty is a stable property of the item that routers cannot read off the prompt. Two experiments ask whether that is fixable.

More data? A learning curve on RouterBench (fixed 20% held-out set, training subsets of 250 to 29,000 items) gives held-out AUC $0.65 \to 0.708$ for MiniLM+logistic; the power-law fit $\text{AUC}(n) = A - Bn^{-c}$ has asymptote $A = 0.741$, with $\text{AUC}(10^6) = 0.726$, or roughly 2.5 doublings of data per +0.01 AUC. Realised gain does improve (+1.47 pp at 8k to +2.10 pp at 29k, $\rho_{\text{realised}} = 0.105$), so data helps, but not nearly fast enough to reach κ. (Exploratory; not pre-registered.)

A stronger router? Table 3 walks up nine router families on the identical 1,500-item held-out split. Nothing escapes the 0.64–0.72 band, and under a paired bootstrap over items with Holm–Bonferroni correction across the four new rungs, *none* of their AUC advantages over the frozen MiniLM+logistic reference is distinguishable from zero (smallest $p_{\text{Holm}} = 0.26$). The largest point estimate is the prompted 70B model with four exemplars, at +0.021 $[-0.019, +0.062]$ against the reference refit here and +0.029 against the 0.6816 of our earlier frozen-representation stage, the baseline the pre-registered criterion names. Either way it is well below the +0.05 fixed in advance as the threshold at which "you tested a weak router" would become a live objection again, so that criterion is NOT MET; nor is the second, since no new rung beats the reference's realised gain by the pre-registered 1.0 pp (the best manages −0.022 pp). One rung answers "maybe the representation is the problem" directly: *unfreezing the encoder* and fine-tuning the same `all-MiniLM-L6-v2` end-to-end gains +0.005 $[-0.014, +0.025]$, with inner-validation AUC 0.751 against 0.694 held out—capacity to fit routing labels that does not transfer, ε made visible.

Now the measurement point: **the highest-AUC rung has the worst matched-cost gain.** The prompted 4-shot router's −1.31 pp is worse than ignoring the query entirely, at the same budget, on the same items. AUC is computed per model and is invariant to any monotone rescaling of that model's scores, but a matched-cost policy ranks items by the *difference* between two models' predicted utilities. A router can therefore order items correctly within each model and still get every cross-model comparison wrong. Since the routing literature reports per-model AUC or accuracy-at-a-budget almost universally, this is a concrete case where the standard metric and the quantity a deployment cares about disagree in sign.

Both at once: the encoder rung at RouterBench scale. The two paragraphs above have a common cause, and repeating the unfreezing experiment with 10× the supervision exposes it. On RouterBench we fine-tuned the same `all-MiniLM-L6-v2` end-to-end on 27,735 training items over all 11 models, and scored it on the 7,299-item held-out split of the learning curve, against the frozen logistic and MLP routers *refit on the same training items* (Table 2; $\kappa = 19.95$ pp on these items; epoch chosen on a 1,460-item inner split that never touches the held-out set).

Unfreezing buys +0.005 AUC here, the same +0.005 it bought on our own data, and **3.4× the matched-cost gain**: it takes ρ_{realised} from 9.7% to 32.6%. An identical AUC delta means nothing in one setting and triples the deployable gain in the other. The frozen MLP makes the point a second time from the other direction: it is 0.046 AUC *worse* than frozen logistic and 0.9 pp *better* at the same budget. Supervision, not capacity, is what changed: the gap between inner-validation and held-out AUC collapses from 0.751 vs 0.694 on our 2,975 training items to 0.7173 vs 0.7126 on 27,735, so the same architecture that merely memorised at our scale generalises at RouterBench's. We therefore state the negative result precisely. What

Table 3: Router-strength ladder on one 1,500-item held-out split. Row 1 summarises four earlier classical rungs on identical data (k-NN 0.6521, gradient boosting 0.6547, a random forest 0.6703 that reaches training AUC 0.9999, logistic regression 0.6816). ΔAUC is a 2,000-resample paired bootstrap over items against the frozen reference, Holm-corrected across the four new rungs; gain is realised matched-cost gain at $\beta = 0.5$ against $\kappa = 4.07$ pp.

Router	Representation	mean AUC	ΔAUC [95%]	gain (pp)
Four classical rungs, earlier stages	MiniLM (frozen)	0.652–0.682	—	—
Logistic regression, refit here	MiniLM (frozen)	0.6896	*reference*	+0.556
Prompted LLM, zero-shot	Llama-3.3-70B	0.6416	-0.048 $[-0.097, +0.004]$	-0.444
Prompted LLM, 4-shot	Llama-3.3-70B	**0.7105**	$+0.021$ $[-0.019, +0.062]$	$\mathbf{-1.311}$
Fine-tuned end-to-end	MiniLM-L6 (unfrozen)	0.6945	$+0.005$ $[-0.014, +0.025]$	+0.533
Fine-tuned end-to-end	MiniLM-L12 (unfrozen)	0.6912	$+0.002$ $[-0.018, +0.023]$	+0.533
LoRA fine-tune	Gemma-3-27B-it	BLOCKED	—	—

nine router families fail to move is **AUC**; what the decomposition shows is that AUC was the wrong thing to watch. On the deployable quantity, at scale, a representation trained for the task recovers roughly a third of complementarity instead of a tenth — so ε is large but *not* immovable, and closing it looks more like a representation-learning problem than a model-capacity one. It is still ε that binds: two thirds of κ remains unclaimed by the best router we could train, with $\rho \approx 1$ saying it is not noise that is in the way.

The random forest is the other diagnostic: it reaches training AUC 0.9999—it memorises the training set perfectly—and still generalises *below* logistic regression. Ample capacity, no held-out gain. That is the signature of a target that is not a smooth function of the input representation, not of an inadequate model class.

What we could not run, and what it costs the argument. The pre-registered top rung was a fine-tuned *generative* LLM router. The LoRA job trained without incident (Gemma-3-27B-it, 3 epochs, 10,500 examples, 1,491,033 tokens, $6.71 booked from the provider's reported price) and then proved unservable through four independent routes, each probed rather than assumed: serverless LoRA inference is unavailable account-wide; dedicated endpoints v1 no longer accept creates platform-wide; the v2 resource model has no certified serving config for this base architecture; and re-training on a base that v2 *can* serve was refused for insufficient account balance. We report the rung as BLOCKED with those errors rather than estimating it, and we state the cost to the argument plainly: a reader who believes a fine-tuned generative LLM router would clear +0.05 has been answered by a fine-tuned encoder, a prompted 70B model, a memorising random forest and a learning curve—not by the thing they asked for.

Two measurement corrections this rests on. Every gain above is measured against a *cost-matched query-independent* baseline and charged *realised per-item* cost, because both choices change conclusions rather than decimals. On our own data a deployed keyword router is beaten on both axes at once by the constant policy "send everything to the mid tier" (86.8% vs 92.3% accuracy at 6.87× the dollar cost), and pricing a router by per-model *mean* cost—standard practice—is biased in the router's favour by a term 1.9× the regret being reported, which flips its sign. Re-auditing RouteLLM [Ong et al., 2025] on its own released data, its router *does* beat a cost-matched mixture, by +0.57 pp: the protocol discriminates rather than rejects. Full statement, derivation and the RouteLLM re-audit are in Appendix J.

6 Related work

RouteLLM [Ong et al., 2025] and FrugalGPT [Chen et al., 2023] report large savings against always-frontier; we re-audit the former against a cost-matched baseline. RouterBench [Hu et al., 2024] supplies the outcome matrix we decompose, but prices policies with per-model mean cost and does not separate complementarity from predictability. Concurrent evaluation work reaches a compatible diagnosis from a different direction—Yuan et al. [2025] and Huang et al. [2025] both argue current router benchmarks and metrics are inadequate,

the former building a larger framework (85 models, 33,337 queries)—but proposes better benchmarks where we propose a decomposition that attributes a shortfall to the model set, the task or the router, and note that the deciding term needs replicate generations no such benchmark provides. The variance machinery is standard (reliability/ICC; CVaR [Rockafellar and Uryasev, 2000]; the mean–variance bound [Bhatia and Davis, 2000]); multiple-comparison control follows Holm [1979] and Benjamini and Hochberg [1995].

7 Limitations

The top rung of the ladder is missing. The pre-registered fine-tuned *generative* LLM router is BLOCKED, for the four provider reasons above. Our strongest evidence against "you tested a weak router" is therefore an end-to-end fine-tuned encoder plus a prompted 70B model, not a fine-tuned generative router, and a reviewer is entitled to hold that objection open. The fine-tuned encoder does at least have the *capacity* to fit the labels (inner-validation 0.751 against 0.694 held out), so on our own data the shortfall is transfer, not capacity. But we do not claim a generative router could not do better on the matched-cost axis: Table 2 shows that axis moving 3.4× while AUC moved +0.005, and an AUC-framed bound is exactly the kind of bound that experiment invalidates.

Null results with wide intervals are not proofs of absence. Every ΔAUC interval in Table 3 is roughly ±0.02–0.05 wide on 1,500 items. We can rule out the +0.05 effect we pre-registered; we cannot rule out a +0.02 one, and we do not.

The encoder-at-scale result is one split, one seed, exploratory. Table 2 is a single 7,299-item held-out set, three epochs, not pre-registered. Its comparison rows are refit on identical items, which controls the comparison but not the sampling variability of the split, and we quote no interval on the 32.6% because we ran it once. Read it as a demonstration that the two axes come apart by a large factor, not as an estimate of what unfreezing buys.

ρ is inferred, not bounded two-sidedly. Our rigorous ceiling is vacuous here, so $\rho \approx 1$ rests on the reliability measurements plus a Beta fit to replicate-identified moments (Proposition 5 needs $\mathbb{E}|\eta - \eta'|$, which $k = 3$ does not identify). That fit is a modelling assumption and is labelled as one throughout.

Scale, and what each half of the evidence cannot do. The replicate study is 48,276 calls over three models, four benchmarks and $48.09 of real API spend — depth without breadth; RouterBench is breadth that cannot replicate. Neither alone suffices and we do not claim the two settings are exchangeable. Both are single-turn, self-contained, auto-gradable academic tasks, so every κ and ρ here is conditional on that item distribution rather than on production traffic. Where earlier stages report joules they mean token counts times assumed per-tier rates, which is why the headline comparisons are stated in measured dollars. Further assumptions — replicate exchangeability, which propositions hold for general K, and the status of the power-law fit — are in Appendix I.

8 Conclusion

Router gains decompose into complementarity, predictability and estimation error, and the three assign blame to the model set, the task and the router respectively. Measured across 11 models and 8 benchmarks, complementarity is abundant (12.11 pp) and routers capture 4.6% of it. We expected predictability to be the missing factor, pre-registered that hypothesis, and refuted it: per-item difficulty is highly reliable, so the information is there. What is missing is the ability to infer it from prompt text. Measured in AUC that gap is stubborn—nine router families, a 70B prompted model and a 29,000-item learning curve all fail to move it. Measured in the quantity a deployment is billed for, it is partly tractable: unfreezing the encoder at RouterBench scale takes the captured share of κ from 9.7% to 32.6% on identical items, for an AUC change of +0.005. Two implications follow, and they point in different directions. For practitioners: the routing metric the field reports does not track the routing value the field wants, so audit against a cost-matched baseline before believing either a gain or a null. For researchers: ε, not ρ, is where the remaining two thirds of κ is lost, and it responds to representation learning—which makes it a better target than the model-capacity scaling that has not helped.

References

Y. Benjamini and Y. Hochberg. Controlling the false discovery rate: a practical and powerful approach to multiple testing. *Journal of the Royal Statistical Society, Series B*, 57(1):289–300, 1995.

R. Bhatia and C. Davis. A better bound on the variance. *The American Mathematical Monthly*, 107(4):353–357, 2000.

L. Chen, M. Zaharia, and J. Zou. FrugalGPT: How to use large language models while reducing cost and improving performance. *arXiv:2305.05176*, 2023.

S. Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.

Q. J. Hu, J. Bieker, X. Li, N. Jiang, B. Keigwin, G. Ranganath, K. Keutzer, and S. K. Upadhyay. RouterBench: A benchmark for multi-LLM routing system. *arXiv:2403.12031*; ICML 2024 workshop, 2024.

Z. Huang et al. RouterEval: A comprehensive benchmark for routing LLMs. *arXiv:2503.10657*; EMNLP, 2025.

I. Ong, A. Almahairi, V. Wu, W.-L. Chiang, T. Wu, J. E. Gonzalez, M. W. Kadous, and I. Stoica. RouteLLM: Learning to route LLMs with preference data. *ICLR*, 2025. *arXiv:2406.18665*.

R. T. Rockafellar and S. Uryasev. Optimization of conditional value-at-risk. *Journal of Risk*, 2(3):21–41, 2000.

J. Yuan, Y. Lu, R. Liu, Y.-N. Chuang, H. Liu, S. Zhong, Y. Sui, G. Wang, J. Xing, and X. Hu. Who routes the router: Rethinking the evaluation of LLM routing systems. *NeurIPS 2025 LLM Evaluation Workshop*, 2025.

A Proofs

Proposition 1 (ladder). The inequalities follow from $\Pi_0 \subset \Pi_1 \subset \Pi_2$: a constant is a function of x, and a function of x is a function of (x, u, c). For concavity, A and C are linear in z and each class is convex and closed under mixing, so if z^1 is feasible at b_1 and z^2 at b_2 then $tz^1 + (1-t)z^2$ is feasible at $tb_1 + (1-t)b_2$ and attains at least $tA(z^1) + (1-t)A(z^2)$. Monotonicity holds because enlarging b enlarges the feasible set. □

Proposition 2 ($\rho = 1$ iff outcomes are deterministic given x). If u, c are $\sigma(x)$-measurable then $\Pi_1 = \Pi_2$ as policy classes with the same value and cost functionals, so $A^\dagger = A^*$ and $\rho \equiv 1$. □

Two-model closed forms. Assume constant per-model costs $c_s < c_l$ and $\mathbb{E}[u_l] \geq \mathbb{E}[u_s]$, and let β be the traffic fraction sent to l, so $b(\beta) = (1-\beta)c_s + \beta c_l$. (a) A query-independent policy mixes the two points and its value is linear and increasing in β, so the budget binds and $S(b) = \mathbb{E}[u_s] + \beta\mathbb{E}[D]$. (b) Maximising $\mathbb{E}[u_s] + \mathbb{E}[z\delta]$ over $z : \mathcal{X} \to [0,1]$ with $\mathbb{E}[z] \leq \beta$ is a fractional knapsack whose solution sets $z = 1$ on the largest values of δ, giving $\beta\mathrm{CTE}_\beta(\delta)$. (c) Identical, with the oracle ranking by realised D. (d) Binary case: any policy has value at most $\mathbb{E}[\max(u_s, u_l)] = 1 - p_{00}$, attained by the cheapest-correct oracle; since $\mathbb{E}[u_s] = p_{11} + p_{10}$ and $\mathbb{E}[u_l] = p_{11} + p_{01}$ we get $(1 - p_{00}) - \mathbb{E}[u_s] = p_{01}$ and $(1 - p_{00}) - \mathbb{E}[u_l] = p_{10}$, so the gain over the better single model is $\min(p_{01}, p_{10})$. □

Since $\delta = \mathbb{E}[D \mid x]$ and CTE_β is a coherent risk measure, conditioning contracts it, which re-proves $\rho \leq 1$ independently of Proposition 1.

Note that $\min(p_{01}, p_{10})$ is *not* $\kappa(b)$: it measures headroom over the better single model, whereas $\kappa(b)$ measures headroom over the matched-cost *mixture*, which is worse than the better single model whenever $\beta < 1$. A router can gain over a mixture even when $\min(p_{01}, p_{10}) = 0$, by spending a fixed budget where it does the most good. We report both because $\kappa(b)$ is the quantity in the decomposition and $\min(p_{01}, p_{10})$ is the quantity a reader intuits.

Proposition 3 (ceiling). In the main text. The variance step is the fact that a $[0,1]$-valued random variable with mean β has variance at most $\beta(1-\beta)$, attained by the two-point $\{0,1\}$ distribution [Bhatia and Davis, 2000]. The maximum of $\sqrt{\beta(1-\beta)}$ over β is at $\beta = 1/2$.

Proposition 4 (identification). The full statement, condensed in the body. Let $\hat{\eta}_m$ be the mean of the k replicates of model m on an item. Then $\mathrm{Var}(\hat{\eta}_m) = \mathrm{Var}(\eta_m) + \frac{1}{k}\mathbb{E}[\eta_m(1-\eta_m)]$ and $\mathbb{E}\big[\frac{k}{k-1}\hat{\eta}_m(1-\hat{\eta}_m)\big] = \mathbb{E}[\eta_m(1-\eta_m)]$, so with $\delta = \eta_l - \eta_s$,

$$\widehat{\mathrm{Var}}(\delta) = \mathrm{Var}(\hat{\eta}_l - \hat{\eta}_s) - \tfrac{1}{k-1}\overline{[\hat{\eta}_l(1-\hat{\eta}_l) + \hat{\eta}_s(1-\hat{\eta}_s)]}$$

is unbiased for $\mathrm{Var}(\delta)$, where the bar denotes the average over items.

Proof. The first identity is the law of total variance with $\mathrm{Var}(\hat{\eta} \mid x) = \eta(1-\eta)/k$. For the second, $k\hat{\eta} \sim \mathrm{Bin}(k,\eta)$ given x, so $\mathbb{E}[\hat{\eta}(1-\hat{\eta}) \mid x] = \frac{k-1}{k}\eta(1-\eta)$ and $\frac{k}{k-1}\hat{\eta}(1-\hat{\eta})$ is unbiased for $\eta(1-\eta)$. Combining across the two models under conditional independence, with $\frac{1}{k}\cdot\frac{k}{k-1} = \frac{1}{k-1}$, gives the stated estimator. □

Proposition 5 (Bayes AUC). Score by η, which is AUC-optimal. With $u = \eta(X)$, $v = \eta(X')$ write $T_> = \mathbb{E}[\mathbf{1}\{u > v\}u(1-v)]$, $T_< = \mathbb{E}[\mathbf{1}\{u < v\}u(1-v)]$, $T_= = \mathbb{E}[\mathbf{1}\{u = v\}u(1-v)]$. Under the half-credit convention the AUC is $(T_> + T_=/2)/(a(1-a))$, and $T_> + T_< + T_= = \mathbb{E}[u(1-v)] = a(1-a)$. By exchangeability $T_< = \mathbb{E}[\mathbf{1}\{u > v\}v(1-u)]$, so $T_> - T_< = \mathbb{E}[\mathbf{1}\{u > v\}(u-v)] = \frac{1}{2}\mathbb{E}|u-v|$. Solving the two equations gives $T_> + T_=/2 = \frac{1}{2}a(1-a) + \frac{1}{4}\mathbb{E}|u-v|$, hence the closed form; the bound follows from $\mathbb{E}|u-v| \le \sqrt{\mathbb{E}(u-v)^2} = \sqrt{2}\,\mathrm{sd}(\eta)$. Sanity checks: η constant gives $1/2$; $\eta \in \{0,1\}$ equiprobable gives 1. □

B Numerical validation of every proposition

Because the propositions are load-bearing, each is checked against a brute-force reference before any result depends on it (`stage11_13/s11_validate.py`, results in `s11_validate.json`). All 14 checks pass.

Table 4: Validation suite. "LP" references are `scipy.optimize.linprog` with HiGHS; others are Monte Carlo.

Check	Reference	Result
$S(b)$ exact	LP over the mixing simplex, 300 cases	max err 0
$A^*(b)$ exact	multiple-choice knapsack LP, 120 cases	max err 0, no budget overrun
Ladder $S \le A^\dagger \le A^*$	Bayes router vs. oracle, n=20k	no violation
$\rho = 1$ iff deterministic	analytic + Bernoulli noise	exact; $\rho \to 0.258$ under noise
Ceiling never violated	exact top-β tail, 4,000 cases	min slack > 0; max tightness 0.856
Ceiling tight	two-point construction	ratio 0.998
Variance components unbiased	known Beta η, 400 cases	rel. bias 0.14%
sd(δ) unbiased	known η pair, 400 cases	rel. bias 0.55%
Naive estimator inflated	same simulation	Var(δ) inflated 2.33×
Bayes AUC closed form	empirical ROC AUC, 200 cases	max err 0.0035
sd bound dominates	empirical AUC*	min slack 0.036
Beta moment match accurate	empirical AUC*	max err 0.0030
$\min(p_{01}, p_{10})$ identity	union minus best single	max err 0
Naive cost axis exact/biased	simulated static vs. router	1.0% vs. 30.3% rel. error

C Pre-registration and deviations

Success criteria, falsification rules, the benchmark-family map, router families, statistics and the cost gate were committed to git *before* the analyses they govern (`stage11_13/prereg_stage11_13.md`); the commit precedes every result artifact and this can be checked against `git log`. This follows the same discipline as two earlier pre-registrations in the project.

Eight deviations are recorded in `stage11_13/DEVIATIONS.md`. The three that affect claims:

D1. H3 ("the router-free bound is $\leq 50\%$ of κ") was **refuted**: the bound is $2.82\times$ κ at median. The pre-registration named this outcome in advance and required us to retreat from the ceiling framing, which §4 does. The replacement claim — difficulty is reliable but not inferable from text — is the one the data supports.

D2. The pre-registration asserted that a policy seeing one replicate $Y^{(1)}$ upper-bounds $A^{\dagger}$. *This is wrong*: $Y^{(1)}$ is a realised outcome, not a function of x, so the two information sets are incomparable; only their join contains $\sigma(x)$. The error was caught while implementing Stage 12, before any result depended on it. The quantity is still reported, relabelled as the unbiased value of a feasible peeking policy rather than a bound.

D8. The pre-registered fine-tuned generative LLM rung is BLOCKED; Appendix F gives the four provider errors. Two additions are exploratory and labelled as such throughout: the learning curves (D3) and the end-to-end fine-tuned encoders (D7). The encoders are judged against the *same* pre-registered +0.05 threshold as the pre-registered rungs, so adding them cannot make the criterion easier to pass.

D Per-family RouterBench detail

Full per-pair, per-family, per-β output is in `stage11_13/s11_routerbench_pairs_0shot.csv.gz` (440 pair-family cells $\times$ 9 operating points, plus the 11-model problem). Summary statistics and all hypothesis verdicts are in `s11_routerbench_0shot.json`. The 5-shot file is analysed identically as a pre-registered replication and reported in `s11_routerbench_5shot.json`.

Multiple-comparison detail at $\beta = 0.5$ for the best router: 49/55 pairs have a positive mean gain, 28 have raw $p < 0.05$, 26 survive Benjamini–Hochberg at FDR 0.05, and 13 survive Holm–Bonferroni at family-wise $\alpha = 0.05$. p-values are two-sided bootstrap over benchmark families with the router and baseline evaluated on identical items.

Table 5: The 5-shot release as a pre-registered replication of the 0-shot primary analysis, run with identical code. Medians over the same 440 pair-family cells at $\beta = 0.5$, 95% bootstrap intervals over cells.

Quantity	0-shot (primary)	5-shot (replication)
items analysed	36,494	36,480 (28 dropped, missing cells)
complementarity κ	12.11 pp [11.15, 12.92]	11.44 pp [10.66, 12.27]
best router	TF-IDF + logistic	TF-IDF + logistic
out-of-fold AUC	0.7160	0.7172
realised gain	+0.585 pp [0.442, 0.767]	+0.640 pp [0.519, 0.790]
ρ_{realised}	4.6% [3.6, 5.9]	5.5% [4.4, 6.8]
pairs with positive mean gain	49/55	50/55
pairs surviving BH / Holm	26 / 13	22 / 11
H1, H2 verdicts	supported	supported

E Cost accounting for the new experiments

The project's earlier stages spent $48.09 of real API budget. Stage 13 was pre-registered with a $25 gate on additional spend, enforced in code: every call accumulates the provider's own reported usage and the run aborts if the gate is crossed (`stage11_13/s13_spend.json`).

The pre-registered gate was never the binding constraint: spend stopped at $9.71 of $25 because the provider refused further fine-tuning jobs, not because we chose to stop. RouterBench, the decomposition, the learning curves, the end-to-end encoder rungs and all numerical validation cost nothing: they analyse already-released outcomes and already-paid-for generations, or are CPU work. The expensive part of this project is the replicate data, and it is the part no public dataset provides.

Table 6: Additional spend for the new experiments. Every line is the provider's own reported usage or job price, accumulated at call time.

Item	Volume	USD
Prompted router, zero-shot (Llama-3.3-70B)	4,500 calls	0.58
Prompted router, 4-shot (Llama-3.3-70B)	4,500 calls	2.41
LoRA fine-tune (Gemma-3-27B-it, 3 epochs)	10,500 examples, 1,491,033 tokens	6.71
Fine-tuned generative router inference	BLOCKED, App. F	0.00
End-to-end fine-tuned encoders (CPU)	2 backbones × 6 epochs	0.00
RouterBench (both releases), curves, theory	—	0.00
Total additional		**9.71**

F The blocked rung, in full

The pre-registration commits us to reporting a blocked run as blocked, with the provider's error, and to estimating nothing in its place. Each route below was probed rather than assumed; verbatim responses are stored in `stage11_13/s13_ftblocked.json` and `stage11_13/s13_endpoint_probe.json`.

1. **The fine-tune itself succeeded.** Job `ft-6567602b-4aa3`, LoRA on `google/gemma-3-27b-it`, 3 epochs over the 10,500 prompt/completion pairs in `s13_ft_train.jsonl` (the exact file is committed), 1,491,033 tokens, $6.71.
2. **Serverless LoRA inference: unavailable.** `/v1/models` advertises 13 `*-Lora` inference targets including `google/gemma-3-27b-it-lora`; every one returns `400 Unable to access non-serverless model` on this account, and the fine-tuned name itself returns `404 model_not_available`.
3. **Dedicated endpoints v1: retired.** `POST /v1/endpoints` returns `403 endpoints_v1_create_access_disabled` — a platform-wide change, not an account limit.
4. **Dedicated endpoints v2: no config for this base.** v2 requires a certified serving config; `models.configs.list` returns *zero* for both the merged fine-tune and the `gemma-3-27b-it` base, and that base is absent from the 43 v2-supported architectures. A `validate_only` deployment create fails for want of a config.
5. **Re-training on a servable base: refused.** `Qwen/Qwen3.5-9B` is fine-tunable *and* has a certified v2 config (1×H100 BF16, $3.99/replica-hour), and it is the Tier-1 model of the system under study, so the router would have cost no more to run than the cheapest tier it routes to. `POST /v1/fine-tunes` returns `402 insufficient_balance`: "Required combined balance and credit limit: 4.00 USD". Pay-as-you-go serverless calls still return 200, so the block is the upfront reserve a fine-tuning job requires, not the credential.

G Reproduction

```
pip install -r requirements-eval.txt
python3 stage11_13/s11_validate.py            # 14/14 proposition checks
python3 stage11_13/fetch_routerbench.py       # download + SHA-256 verify
python3 stage11_13/s11_routerbench.py 0shot   # decomposition, 55 pairs x 8 families
python3 stage11_13/s12_ceiling.py             # replicate variance components
python3 stage11_13/s11_routerbench.py 5shot   # pre-registered replication
python3 stage11_13/s12b_learning_curve.py     # learning curves
python3 stage11_13/s13b_encoder_finetune.py   # end-to-end encoder rungs, CPU, $0
python3 stage11_13/s13c_encoder_routerbench.py 0shot   # same, on RouterBench
python3 stage11_13/s13_llm_router.py probe    # cost projection before spending
python3 stage11_13/s13_llm_router.py report   # C1/C2 verdicts and bootstrap CIs
```

Seeds, package versions, model strings, endpoints and dataset revisions for all stages are listed in `stage7_10/reproducibility_manifest.md` and `stage11_13/`. Reproducibility hazards are flagged rather than glossed: no provider exposes an immutable model revision, so exact regeneration of any API-derived number is guaranteed by nothing under our control. The RouterBench pickle is pinned by SHA-256 in the result JSON, and `fetch_routerbench.py` refuses to proceed on a mismatch.

H Replicate study detail

Table 7: Replicate-based decomposition, $k = 3$. *Reliability* is the share of per-item outcome variance that is stable between-item signal rather than regeneration noise. AUC* is the Proposition 5 point estimate under a Beta fit to the replicate-identified moments; the distribution-free upper bound of Proposition 5 is 1.0 in all six cells and is therefore uninformative here, which is why the Beta fit is quoted and labelled as a modelling assumption.

		Tier 1 (9B)		Tier 2 (20B)		Tier 3 (GPT-4o)	
Set	n	reliab.	AUC*	reliab.	AUC*	reliab.	AUC*
pool, T=0.7	5,000	0.895	0.992	0.873	0.987	0.942	0.997
test, T=0.0	364	0.772	0.948	0.902	0.992	0.983	1.000

The pool set is 5,000 prompts × 3 models × $k = 3$ at temperature 0.7 (45,000 calls); the test set is 364 prompts × 3 models × $k = 3$ at temperature 0 (3,276 calls). Both are drawn from HumanEval, MBPP, MMLU and GSM8K with execution- or exact-match grading, so no model sits in the grading loop. Temperature 0 is *not* deterministic on either provider, which is what makes the T=0 column meaningful rather than degenerate: the 9B tier changes its graded verdict on 19.8% of items between identical temperature-0 calls.

I Further limitations and assumptions

The limitations a reviewer most needs are in the body; these are the remaining assumptions, stated so they can be checked rather than taken on trust.

Replicate exchangeability. Proposition 4 assumes replicates are conditionally i.i.d. given the item and independent across models. Positive within-item correlation — for instance a prompt that reliably pushes a model into the same wrong reasoning path — would understate the noise term and hence *overstate* sd(δ) and the ceiling. The error therefore runs in routing's favour, so it cannot manufacture our negative direction; it could however make the vacuousness of Proposition 3 look worse than it is.

Which propositions are general. Propositions 1, 2, 4 and 5 hold for arbitrary K. Proposition 3 and the closed-form frontier expressions are two-model statements; with $K > 2$ we compute the frontiers by linear program instead (§B checks the two agree at $K = 2$).

The learning curve is a fit, not a bound. AUC(n) $= A - Bn^{-c}$ is fitted by least squares on nine training sizes over roughly two decades of n. Its asymptote $A = 0.741$ is an extrapolation whose functional form we chose; a different form with the same in-range fit can put the asymptote elsewhere. We use it to argue that the observed trend is slow, not to prove a ceiling.

Three models is not a model ecosystem. The reliability numbers come from one small reasoning model, one mid-tier model and GPT-4o. They should be re-measured before being assumed for a different tier structure, and nothing in this paper establishes that reliability is stable across model families.

J Matched-cost baselines and the biased cost axis

The two measurement corrections summarised at the end of §5, in full. Both are prior stages of this project rather than contributions of this paper, but every number in §3–5 depends on them.

The baseline. On our own 364-item frozen test set, a deployed keyword router is beaten on *both* axes simultaneously by the constant policy "send everything to the mid tier": 86.8% vs 92.3% accuracy at 6.87× the measured dollar cost. Its headline saving against always-frontier is 88.6% in modelled joules but 55.0% in measured dollars—the choice of cost axis moves the number by 4× on identical routing decisions. Applying the same audit to RouteLLM [Ong et al., 2025] on its own released GSM8K data, its router *does* beat a cost-matched query-independent mixture, by a mean of +0.57 pp: positive at 8 of 9 operating points (sign test $p = 0.039$) but significant at none individually ($n = 1{,}307$). So the protocol discriminates rather than rejects— and even the passing case has an effect the field's benchmarks cannot resolve point-by-point, while being reported as "up to 85% cost reduction".

The axis. Pricing a routing policy by multiplying traffic fractions by per-model *mean* cost is exact for a query-independent policy but biased for a router, because routers escalate precisely the items that emit more tokens. The exact correction is

$$R_{\text{true}} = R_{\text{naive}} + \sum_{i \neq j} \operatorname{Cov}\big(\mathbf{1}\{t^*{=}i, \hat{t}{=}j\},\, e_j(x) - e_i(x)\big).$$

On our data this term is 1.9× the regret being reported and flips its sign, reconciling to 2×10^{-16}; on RouteLLM's own GSM8K data the sign flip reproduces and naive accounting overstates savings by up to 6.3%. The bias always favours the router and grows with within-model cost dispersion, so it worsens as the field routes among reasoning models with variable-length thinking budgets. All routers in this paper are charged realised per-item cost.

Note that this correction is the *same covariance object* as Proposition 3: routing lives or dies on one covariance between a routing indicator and a per-item difference, read once on the utility axis and once on the cost axis.